

FULL STACK DEVELOPMENT

FASE 2 | AULA 04 -

GRAPHQL COM NODEJS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	11
REFERÊNCIAS.....	12

EMAND

O QUE VEM POR AÍ?

Nesta aula, aprenderemos o que é o GraphQL e veremos como ele pode ajudar significativamente na integração dos nossos sistemas. Exploraremos os conceitos e práticas fundamentais, incluindo a história e origem do GraphQL, como ele resolve problemas comuns encontrados em APIs REST, e como realizar consultas específicas que retornam exatamente os dados necessários.



HANDS ON

Nesta aula, aprenderemos sobre o GraphQL e realizaremos atividades práticas para fixar o conteúdo. Vamos lá?

EXEMPLO

SAIBA MAIS

GraphQL é uma linguagem de consulta para APIs e um tempo de execução para executar essas consultas por meio de seus dados existentes.

Ele foi desenvolvido internamente pelo Facebook em 2012 e lançado ao público em 2015. GraphQL permite aos clientes solicitar exatamente os dados de que precisam e nada mais, o que torna as interações entre clientes e servidores mais eficientes e flexíveis.

Na figura 1 temos uma demonstração do fluxo do GraphQL. Note que do lado esquerdo nós temos algumas devices que, através de consultas com o GraphQL, conseguem resultados de 3 bases de dados distintas.

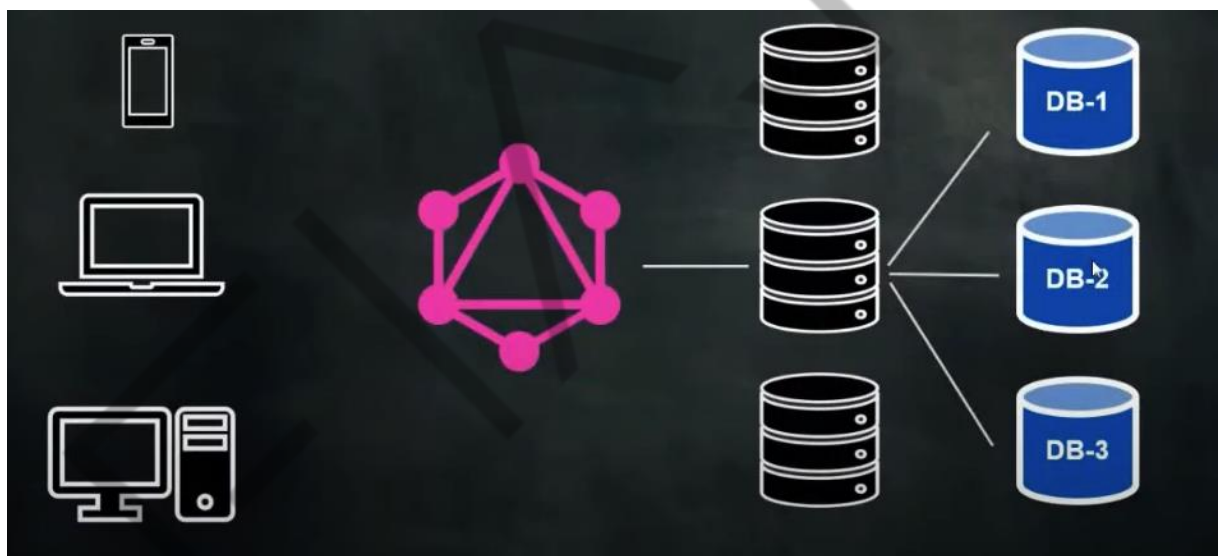


Figura 1 - Exemplo de fluxo com GraphQL
Fonte: elaborado pelo autor (2022)

O GraphQL foi criado para enfrentar os desafios de desempenho e flexibilidade que surgiram com o aumento da complexidade das aplicações móveis e web. Antes dele, muitas aplicações usavam REST para comunicação entre cliente e servidor.

No entanto, as APIs REST tradicionais muitas vezes resultavam em super-fetching (buscando mais dados do que o necessário) ou under-fetching (não buscando dados suficientes), levando a múltiplas requisições para obter os dados necessários.

Esse foi um dos motivadores que me induziram ao estudo da linguagem de consulta na época. Trabalhava em um projeto para a TV Bandeirantes que tinha uma versão web e uma mobile e, na época, tivemos algumas reclamações por conta do payload que era enviado para o app mobile. A seguir, na figura 2, temos o payload utilizado na época.



Figura 2 - Exemplo de payload de projeto Band
Fonte: elaborado pelo autor (2022)

Analisando esse retorno, ele está tranquilo, certo? Mas imagine que, de todas essas informações, o nosso app mobile precisava apenas de duas ou três, e as outras eram perdidas no mobile, sendo utilizada apenas na versão web. Bom, e como resolvemos isso com o GraphQL?

Antes de mostrarmos como resolvíamos na época, vamos entender a sua estrutura. O GraphQL possui dois principais componentes: Queries e Mutations.

Iniciando pelas Queries, elas são usadas para buscar dados. Com uma query GraphQL, você pode solicitar exatamente os campos e objetos de que precisa.

Aqui está um exemplo de uma query simples:

```
{
  match(id: "2") {
    id
    championshipId
    championshipName
    hour
    stadium
    homeTeam {
      name
      shield
      initials
    }
    awayTeam {
      name
      shield
      initials
    }
    homeScore
    awayScore
    status
    penalties
    homePenaltyScore
    awayPenaltyScore
  }
}
```

Esta query busca os detalhes de uma partida específica usando seu ID.

Já as mutations são utilizadas para modificar dados. Elas permitem criar, atualizar ou deletar dados no servidor.

Aqui está um exemplo de uma mutation que cria um novo usuário:

```
mutation {
  updateScore(matchId: "2", homeScore: 1, awayScore: 0) {
    id
    homeScore
    awayScore
  }
}
```

Esta mutation atualiza o placar de uma partida específica.

Agora, para ficar mais claro, vejamos como criar um projeto simples utilizando Node.js e GraphQL.

Escolha um diretório no seu computador e em seguida execute os comandos abaixo dentro de um terminal:

```
cd seu-diretorio  
npm init -y  
npm install apollo-server graphql
```

Crie um arquivo chamado schema.graphql na raiz do projeto e defina o schema GraphQL:

```
type Team {  
  name: String!  
  shield: String!  
  initials: String!  
}  
  
type Match {  
  id: ID!  
  championshipId: Int!  
  championshipName: String!  
  hour: String!  
  stadium: String!  
  homeTeam: Team!  
  awayTeam: Team!  
  homeScore: Int!  
  awayScore: Int!  
  status: String!  
  penalties: Boolean!  
  homePenaltyScore: Int!  
  awayPenaltyScore: Int!  
}  
  
type Query {  
  match(id: ID!): Match  
  matches(championshipId: Int!): [Match]  
}  
  
type Mutation {  
  updateScore(matchId: ID!, homeScore: Int!, awayScore: Int!): Match  
  updateStatus(matchId: ID!, status: String!): Match  
}
```

Em seguida, crie um arquivo chamado resolvers.js e implemente os resolvers para as queries e mutations definidas no schema:


```
const matches = {
  "2": {
    id: "2",
    championshipId: 674,
    championshipName: "Série B 19",
    hour: "20:00",
    stadium: "Arena Pantanal",
    homeTeam: {
      name: "Cuiabá",
      shield: "https://s3-sa-east-
1.amazonaws.com/logos.footstast.net/88x88/cuiaba.png",
      initials: "CUI"
    },
    awayTeam: {
      name: "Sport",
      shield: "https://s3-sa-east-
1.amazonaws.com/logos.footstast.net/88x88/sport.png",
      initials: "SPT"
    },
    homeScore: 0,
    awayScore: 0,
    status: "PartidaNaoiniciada",
    penalties: false,
    homePenaltyScore: 0,
    awayPenaltyScore: 0
  }
};

const resolvers = {
  Query: {
    match: (parent, args) => matches[args.id],
    matches: (parent, args) => Object.values(matches).filter(match =>
match.championshipId === args.championshipId)
  },
  Mutation: {
    updateScore: (parent, args) => {
      const match = matches[args.matchId];
      if (match) {
        match.homeScore = args.homeScore;
        match.awayScore = args.awayScore;
      }
      return match;
    },
    updateStatus: (parent, args) => {
      const match = matches[args.matchId];
      if (match) {
        match.status = args.status;
      }
      return match;
    }
  }
};
```

```
}  
}  
};  
  
module.exports = resolvers;
```

Crie um arquivo chamado `server.js` para configurar e iniciar o servidor GraphQL:

```
const { ApolloServer, gql } = require('apollo-server');  
const fs = require('fs');  
const path = require('path');  
const resolvers = require('./resolvers');  
  
const typeDefs = gql(fs.readFileSync(path.join(__dirname, 'schema.graphql'),  
'utf8'));  
  
const server = new ApolloServer({ typeDefs, resolvers });  
  
server.listen().then(({ url }) => {  
  console.log(`🚀 Server ready at ${url}`);  
});
```

Agora, execute o servidor GraphQL: `node server.js` e abra no seu navegador o seguinte endereço: <http://localhost:4000/>. Nessa url você terá um sandbox onde poderá testar o seu projeto.

A seguir, na figura 3, temos um exemplo de uma query neste projeto dos passos anteriores.

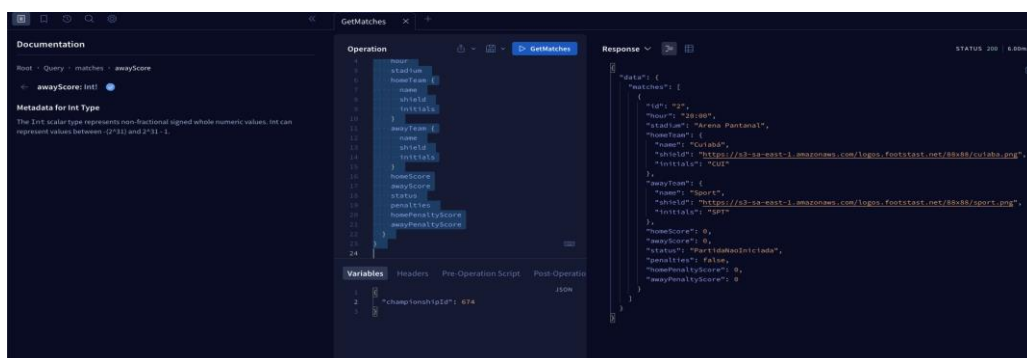


Figura 3 - Exemplo de query graphQL
Fonte: elaborado pelo autor (2024)

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, nós aprendemos o que é o GraphQL e vimos como ele pode ajudar muito na integração dos nossos sistemas.

EXEMPLO

REFERÊNCIAS

Construindo uma API GraphQL com Node.js. **Imasters**. 2019. Disponível em: <https://imasters.com.br/back-end/construindo-uma-api-graphql-com-node-js>. Acesso em: 09 ago. 2024.

Introdução ao GraphQL. **Medium**. 2018. Disponível em: <https://programadriano.medium.com/introdu%C3%A7%C3%A3o-ao-graphql-9f09b33550e7>. Acesso em: 09 ago. 2024.

Qual é a diferença entre GraphQL e REST? **AWS**. 2024. Disponível em: <https://aws.amazon.com/pt/compare/the-difference-between-graphql-and-rest/>. Acesso em: 09 ago. 2024.

PALAVRAS-CHAVE

Palavras-chave: GraphQL, Query, Mutations, Resolvers.

EXEMPLO